Module 3 — Project Context with `CLAUDE.md`

Claude Code Bootcamp · Day 1 · Block 3 of 10

Promise

In 22 minutes you will:

1. Understand what `CLAUDE.md` is and why it sits at repo root.
2. Author a `CLAUDE.md` for **a real repository of your choice**.
3. Watch Claude follow your conventions on the very next prompt.

Why this matters

- Without a `CLAUDE.md`, you re-explain your stack, conventions, and "do nots" in every prompt. That is wasted typing and inconsistent output.
- A good `CLAUDE.md` is the difference between *"please use Python 3.11 with type hints"* repeated 30 times and *"add the endpoint"*.
- It is the single most leveraged file in an AI-paired codebase.

Concepts

- `CLAUDE.md` : a project-level instruction file Claude Code reads automatically on every prompt.
- **Behavior file, not a doc**: every line should change Claude's output. If a line is just nice-to-know, it belongs in `README.md` .
- **Sections that pay rent**: Stack · Conventions · Commands · Do-not list · Glossary.
- **The trim test**: delete each section in turn; if Claude still behaves the same, that section was bloat.

Live demo flow

1. Instructor opens the repo from module 2's lab — no `CLAUDE.md` yet.
2. Asks Claude: *"Add an `--export csv` flag."* Observe the output.
3. Adds a 12-line `CLAUDE.md`. Asks the same prompt in a fresh chat.
4. Class sees the diff: same intent, on-convention output.
5. Run the trim test live — delete one section, re-prompt, observe drift.

Mini project

`CLAUDE.md` for a real repo.

- Pick the repo from module 2 (or any personal repo you trust to commit to).
- Author `CLAUDE.md` covering Stack · Conventions · Commands · Do-not.
- Keep it under 80 lines — every line earns its place.
- Commit it.

Step-by-step lab

1. `cd` into the repo from module 2 (or your repo of choice).
2. Open `skills/claude-md-template/SKILL.md` for a worked example.
3. Run the prompt below; let Claude draft a candidate.
4. Edit ruthlessly. Delete anything that doesn't change behavior.
5. Save as `CLAUDE.md` at repo root and commit.
6. Open a fresh Claude Code chat and ask one new prompt about the repo. Verify on-convention output.
7. Copy the final file into your submission as `module-03/CLAUDE.md` plus a screenshot of Claude obeying it (`module-03/proof.png`).

Suggested Claude Code prompts

```
You are drafting CLAUDE.md for the repo at the current working directory.
Read the repo first. Then propose a CLAUDE.md with these sections:

# Stack       — languages, package managers, runtime versions
# Conventions — naming, file layout, lint/format rules
# Commands    — exact commands for build, test, run, lint
# Do-not      — things you must never do (e.g., add deps without asking)
# Glossary    — domain terms only this team uses

Each line must change your behavior on a future prompt. If a line is just
documentation, omit it. Keep the whole file under 80 lines.
```

Deliverable checklist

- [] `module-03/CLAUDE.md` exists and is < 80 lines.
- [] The file has all five sections — `Stack`, `Conventions`, `Commands`, `Do-not`, `Glossary`.
- [] `module-03/proof.png` shows Claude following one convention from the file.
- [] You can name one line you deleted during the trim test, and why.

Definition of done

✅ `CLAUDE.md` committed to the underlying repo · ✅ Trim test performed at least once · ✅ Submission folder populated.

Review checkpoint

Pair (60 s each):

1. Read the partner's `CLAUDE.md`.
2. Find one line that does *not* change Claude's behavior. Defend or delete.
3. Confirm the proof screenshot actually demonstrates obedience.

Common mistakes

- Treating `CLAUDE.md` as an `ABOUT.md`. It is not — it is a behavior file.
- Stuffing 200 lines in. Long files dilute attention; keep ≤ 80.
- Forgetting the **Do-not** section. This is where you encode hard-won lessons.
- Not committing it. If it isn't in version control, it isn't real.

Instructor notes

- 5 / 5 / 10 / 2 split.
- Open `skills/claude-md-template/SKILL.md` live — students copy from it.
- For students with no personal repo, point them at `module-02/`.
- If short, drop the proof screenshot but keep the file.

Transition to next module

We have *one* prompt working well, and it is now context-aware. Next we generate **multiple candidates** and pick the winner — turning Claude from a single proposer into a small team.

Next: Module 4 — Build Faster with Best-of-N.